

Nyaya Saathi — Architecture Documentation

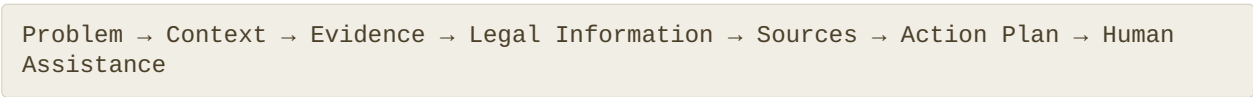
Version: 1.0 (Hackathon / Pilot Build) **Audience:** Engineers, technical reviewers, and judges evaluating the system design.

1. Overview

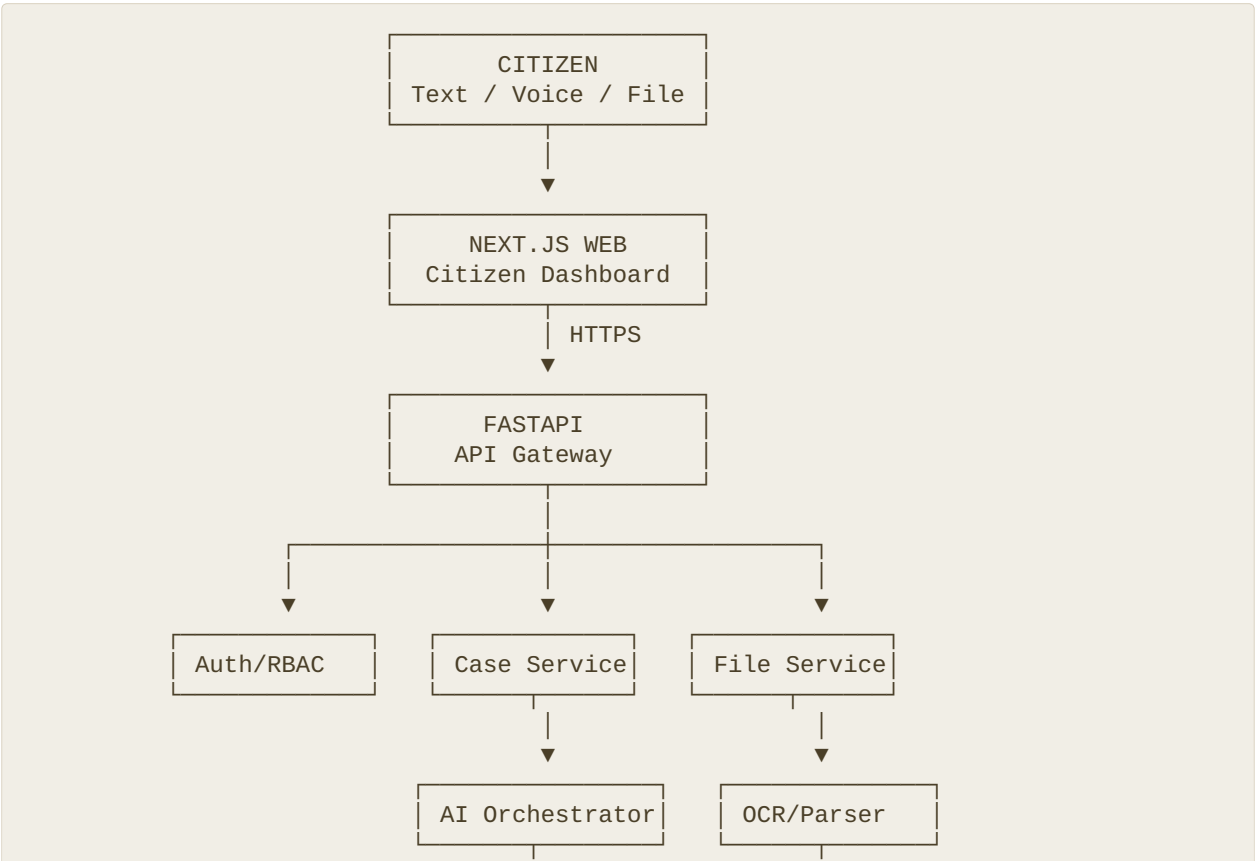
Nyaya Saathi is an AI-assisted legal navigation platform. It turns a citizen's plain-language description of a legal problem into a structured case, grounds its explanations in a curated, authoritative legal knowledge base (via RAG — retrieval-augmented generation), and produces a personalised action plan, evidence checklist, and optional document draft — while pointing the user to official portals and human/legal assistance where appropriate.

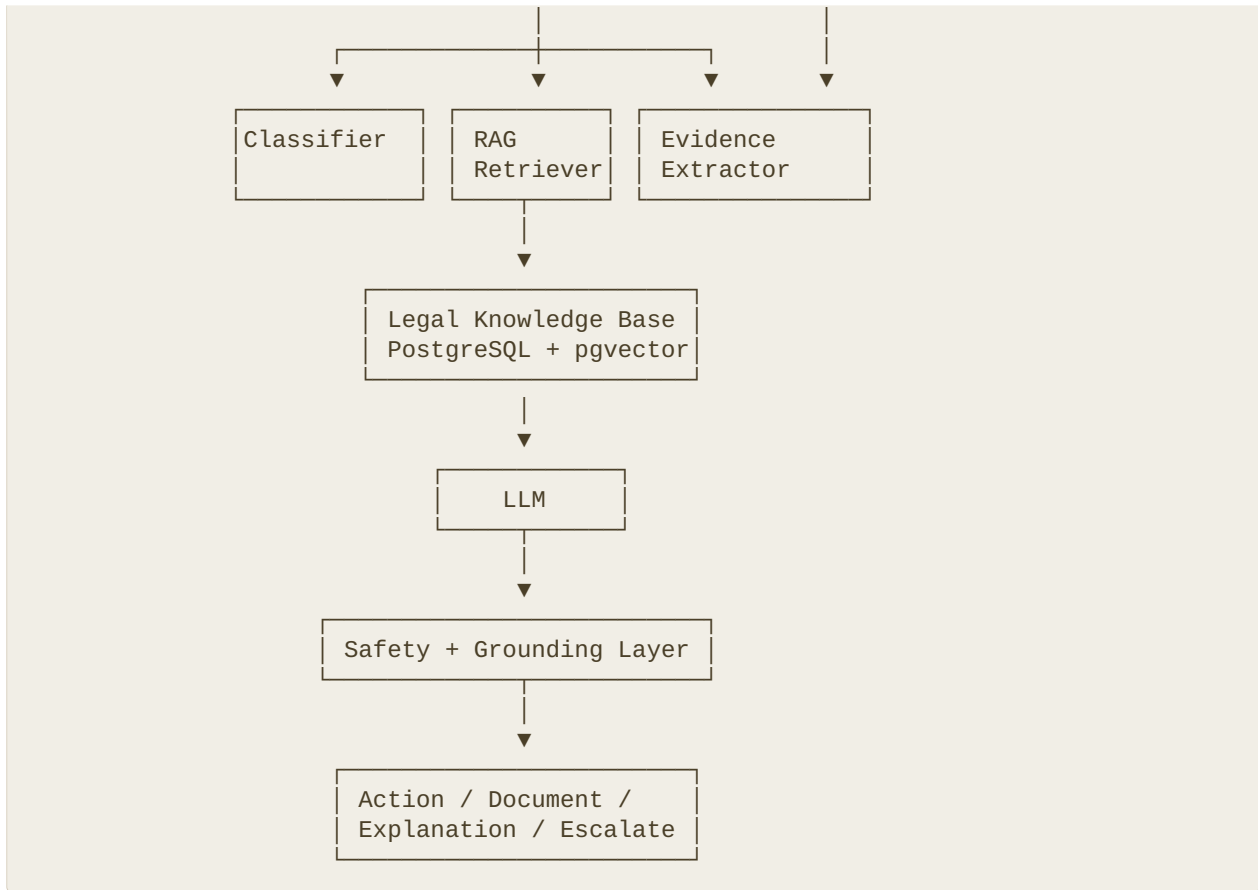
The system is explicitly **not** an autonomous lawyer. It is a navigation and information layer that sits in front of the real legal system.

Core product loop:



2. High-Level Architecture





Layers

Layer	Responsibility
Citizen Interface	Account creation, language selection, issue description, follow-up Q&A, evidence upload, viewing summaries/sources/action plan/documents, case tracking.
Case Intelligence	Converts unstructured input into category, subcategory, entities, dates, amounts, parties, jurisdiction, urgency, evidence inventory, missing information, recommended workflow.
Legal Knowledge + RAG	Retrieves relevant chunks from a curated, versioned legal knowledge base instead of letting the LLM answer from memory alone.
Action & Assistance	Produces the issue assessment, supporting information, missing evidence, next steps, document drafts, and — where appropriate — human/official escalation.

3. Technology Stack

Domain	Technology
Frontend	Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Zod
Backend API	FastAPI, Python, Pydantic, SQLAlchemy, Alembic
Database	PostgreSQL + pgvector
AI / RAG	LLM API, embedding model, hybrid retrieval, optional reranker, structured outputs
Document Intelligence	PyMuPDF, OCR engine, MIME/file validation, entity extraction
Infrastructure	Docker, CI/CD (GitHub Actions), managed PostgreSQL, cloud object storage, HTTPS
Observability	Structured logging, request IDs, error tracking, product/technical/AI metrics

4. Repository Structure

```
nyaya-saathi/
├── apps/
│   ├── web/                # Next.js frontend (app, components, hooks, lib, services,
types, tests)
│   └── api/                # FastAPI backend (core, api, models, schemas, services,
repositories, workers, security)
├── packages/
│   ├── prompts/           # Shared prompt templates
│   ├── shared-types/      # Types shared between frontend/backend
│   └── ui/                 # Shared UI components
├── ai/
│   ├── ingestion/         # Legal source ingestion pipeline
│   ├── retrieval/         # RAG retrieval logic
│   ├── classification/    # Issue classifier
│   ├── evidence/          # Evidence extraction
│   ├── generation/        # Action-plan / document generation
│   ├── evaluation/        # RAG and classifier evaluation
│   └── prompts/           # System / task prompts
├── data/
│   ├── raw/               # Raw ingested legal sources
│   ├── processed/         # Normalised, chunked sources
│   └── manifests/         # Source metadata manifests
├── docs/                  # This documentation package
├── infra/                 # Docker + deployment configuration
├── scripts/               # seed.py, ingest.py, evaluate.py
└── .github/workflows/     # CI/CD pipelines
```

5. Data Model

All tables use UUID primary keys and `created_at` / `updated_at` timestamps where relevant.

users

Column	Type
id	UUID (PK)
email	VARCHAR, unique
password_hash	TEXT
name	VARCHAR
preferred_language	VARCHAR
role	VARCHAR (USER , ADMIN , future: LEGAL_REVIEWER)

cases

Column	Type
id	UUID (PK)
user_id	UUID → users.id
title	VARCHAR
description	TEXT
category / subcategory	VARCHAR
jurisdiction	VARCHAR
urgency	VARCHAR
classification_confidence	NUMERIC
status	VARCHAR

case_questions

Column	Type
id	UUID (PK)
case_id	UUID → cases.id
question / answer	TEXT

evidence

Column	Type
id	UUID (PK)
case_id	UUID → <code>cases.id</code>
original_filename	TEXT
storage_key	TEXT
mime_type	VARCHAR
file_size	BIGINT
extracted_text	TEXT
metadata	JSONB
processing_status	VARCHAR

legal_sources

Column	Type
id	UUID (PK)
title / authority	TEXT
source_type	VARCHAR
jurisdiction / language	VARCHAR
source_url	TEXT
version	VARCHAR
effective_from / effective_to	DATE
checksum	VARCHAR

legal_chunks

Column	Type
id	UUID (PK)
source_id	UUID → <code>legal_sources.id</code>
section_label	TEXT
content	TEXT
embedding	VECTOR
metadata	JSONB

`retrieval_logs`, `action_plans`, `generated_documents`, `audit_logs`

Track retrieval queries and latency, generated action-plan steps, generated document content/versions, and privileged operations respectively. See `SECURITY.md` for what is deliberately **excluded** from these logs.

6. RAG Pipeline

Official Source → Download/Collect → Normalize → Extract Text → Remove Boilerplate
→ Split into Legal Chunks → Attach Metadata → Generate Embeddings → Store in pgvector

- **Chunking** is structure-aware where possible (Act → Chapter → Section → Subsection; or Title → Heading → Subheading → Paragraph), and every chunk retains its document title, section, authority, jurisdiction, source, and version.
- **Retrieval** is hybrid: semantic vector search + keyword search + metadata filters (jurisdiction, domain, source authority, language, active/superseded status).
- **Reranking**: top 10–20 candidates retrieved, reranked down to the 3–6 strongest sources actually passed to the LLM.
- **Generation** input: system rules + user case + retrieved sources + evidence summary + jurisdiction. Output: issue summary, relevant information, evidence observed, missing information, action plan, cautions, and sources.
- **Source governance**: every knowledge-base document carries metadata (`source_id`, `title`, `source_type`, `authority`, `jurisdiction`, `language`, `effective_from/to`, `retrieved_at`, `source_url`, `checksum`, `version`). Source updates are never silent — they are re-hashed, re-versioned, re-embedded, and the old content is marked superseded.
- **Source hierarchy**: Tier 1 (official Acts/portals/courts/legal-aid authorities) is authoritative; Tier 2 (official institutional guidance) is supporting; Tier 3 (secondary material) is used only for explanation and is clearly labelled — it never silently overrides a primary source.

Initial knowledge domains: cyber/online fraud, consumer complaints, rental/tenant issues, employment/wage issues, banking/financial grievance, basic document/contract disputes, and legal-aid navigation.

7. AI Orchestrator

The orchestrator is the single coordinator for a case's AI workflow:

Case Input → Normalize → Classify → Determine Missing Information
→ Retrieve Legal Sources → Analyze Evidence → Generate Explanation
→ Generate Action Plan → Generate Document Draft
→ Safety / Grounding Check → Persist Result

Structured (schema-validated) outputs are used wherever possible instead of parsing free-form prose, to keep the pipeline reliable and testable.

8. Safety Architecture

- The system prompt instructs the model to use **only retrieved sources** for legal claims, and to say so — rather than invent an answer — when sources don't support a claim.
- The model never fabricates section numbers, case law, deadlines, authorities, fees, or procedures.
- Output distinguishes: facts the user provided, information extracted from evidence, retrieved legal information, and model-generated suggestions.
- High-risk / urgent situations (e.g. active fraud, safety risk) prioritise immediate safety guidance and escalation to the appropriate official channel over a general explanation.
- All material legal claims carry a source reference.

See `SECURITY.md` for the full threat model (prompt injection, RAG poisoning, malicious uploads, unauthorized access, API abuse) and mitigations.

9. Scalability & Performance Notes

- Stateless API layer behind the FastAPI gateway allows horizontal scaling.
 - Vector search via pgvector scales with proper indexing (e.g. IVFFlat/HNSW) and metadata filtering to keep candidate sets small.
 - Long-running work (OCR, embedding generation, document generation) is designed to run as background jobs rather than blocking request/response cycles.
 - Caching is used for frequently retrieved legal chunks and static reference data.
 - Streaming AI responses improve perceived performance for chat-style interactions.
-

10. Observability

- Structured logs with request IDs across frontend → API → AI orchestrator.
 - Technical metrics (latency, error rate, throughput) and product metrics (cases created, completion rate of the workflow) are tracked separately from AI metrics (classification confidence distribution, retrieval latency, citation rate, abstention rate).
 - Health (`GET /health`) and readiness (`GET /ready`) endpoints support deployment monitoring.
-

11. Related Documents

- `API.md` — full REST API reference.
 - `USER_GUIDE.md` — end-user walkthrough of the product.
 - Security threat model and data-handling detail live in the platform's Privacy Policy and Terms & Conditions.
-